EE1007
Introduction to Computer Science Laboratory

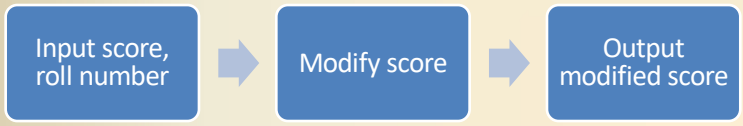
Lab 10 Vector

Andy, Yu-Guang Chen
Assistant Professor, Department of EE
National Central University
andyygchen@ee.ncu.edu.tw

1

Lab 10.1
P113033. Modifying Score (30%)

- Test target: Fundamental operation of vector
- Use std::vector to store the input scores and manipulate a specific student's score.
- Output error message if the input roll number k is out of boundary.
- **Notice the range of k.**
- Flow chart:



2

Lab 10.1
P113033. Modifying Score (30%)

- Pseudo code of an example of modifyScore:

```
function modifyScore(array, k):
   // check if k is within valid bounds
   if 0 < k <= size of array then
      if array[k-1] <= 90 then
         array[k-1] = array[k-1] + 10
      else
         array[k-1] = 100
      endif
   else
      print "invalid student number."
   endif
end function
```


Lab 10.2
P113034. Transpose Matrix (30%)

- Test target: Use of 2D array/vector, nested loop.
- Declare a 2D array with sufficient room to store the matrix, i.e., 100x100.
- Transpose the matrix by nested loop.
- Flow chart:



Lab 10.2
P113034. Transpose Matrix (30%)

- An example of transpose:

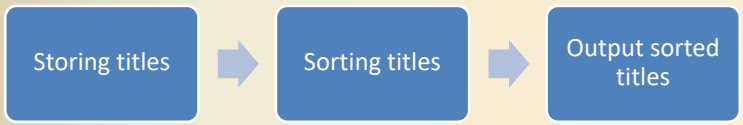
```
1  void transpose(int matrix[][100], int result[][100], int n, int m) {
2      for(int i=0; i<n; i++){
3          for(int j=0; j<m; j++){
4              result[j][i] = matrix[i][j];
5          }
6      }
7  }
```

- Notice the bracket of 2D array when passed to a function as an argument.

5

Lab 10.3
#P113035. Book Arrangement (30%)

- Test target: Use of array, basic concept of sorting.
- This problem requires a well-structured flow, so it's recommended to perform paperwork before you really working on your code.
- There is more than one way to solve this problem, however, it can be breakdown to these part: Storing book titles, Sorting titles, Output titles.



6

Lab 10.3
#P113035. Book Arrangement (30%)

- Storing titles
  - First, you need to choose the data structure to store the titles. Here are some examples.
    1. Use 2D int array to store the ASCII value of each character of the word.
    2. Use char array to perform C-type string operations.
    3. Use 2D vector to store the value, which shares the same concept with method 1.

```
1  int titles[100][100] = {0};
2  char titles[100][100] = {0};
3  vector<vector<int>> titles(n);
```


7

Lab 10.3
#P113035. Book Arrangement (30%)

- Storing titles – int array
  - If you're using int array, you can receive the letter by while loop with an end-checking condition.

```
1  void inputTitles(int titles[][100], int n) {
2      cin.ignore(); // Ignore the newline after the number of books
3      for (int i = 0; i < n; ++i) {
4          int index = 0;                  // Position in the current title row
5          char input_character = '\0';    // Character buffer
6
7          while ((input_character = cin.get()) != '\n') {  // Check for newline
8              titles[i][index++] = input_character;        // Store the character in the array
9          }
10         titles[i][index] = 0;               // Stands for the end of the word
11     }
12 }
```


8

Lab 10.3
#P113035. Book Arrangement (30%)

- Sorting titles
  - Secondly, you can use any method (e.g. insertion sort, bubble sort, radix sort) to sort the titles, as there are no restrictions on sorting.
  - Compares the titles by lexicographic order.
  - Notice that this problem requires you to perform **case-insensitive comparation**, so you should convert all the letters into lowercase or uppercase when performing comparation.

Lab 10.3
#P113035. Book Arrangement (30%)

- Sorting titles – an example of bubble sort

```
function bubblesort(titles):
   n = size of titles
   for i from 0 to n – 2 do
         for j from 0 to n – i – 2 do
         need_swap = compare(titles[j], titles[j + 1]))
         if (need_swap) then
            swap titles[j] with titles[j + 1]
         end if
      end for // end of inner loop
   end for // end of outer loop
end function
```

case-insensitive comparation

Lab 10.4
#P113036. Decrypting Message (20%)

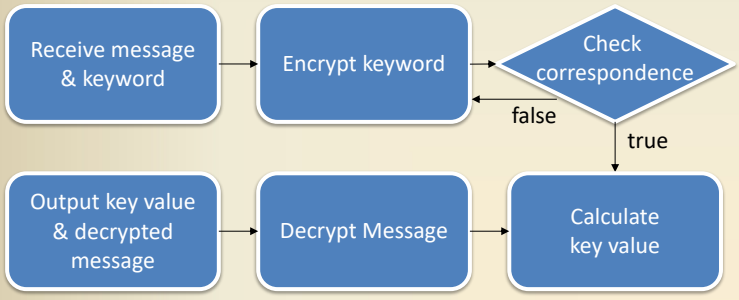
- Test target: Use of array.
- This problem gives you an encrypted message and a keyword, you need to search the position of the keyword and calculate the corresponding key value.
- Once you solve the previous problem(by yourself), this problem should be easy for you!
- One corner case is when the key value = 26. The program should output the result as 26(not 0).

▪ The first line should contain a single integer key, representing the Caesar Cipher shift that was applied to word to produce the embedded phrase in message, notice that 1 ≤ key ≤ 26.

11

Lab 10.4
#P113036. Decrypting Message (20%)

- An example flow to solve this problem is given below:



12

Evaluation

- Lab 10.1(30%)
  - 5 testcases, 6% each.
- Lab 10.2(30%)
  - 5 testcases, 6% each.
- Lab 10.3(30%)
  - 5 testcases, 6% each.
- Lab 10.4(20%)
  - 5 testcases, 4% each.

13





Andy, Yu-Guang Chen

Assistant Professor, Department of EE, NCU

Email: andyygchen@ee.ncu.edu.tw

14